

Conteos certificados de ciclos cortos para los códigos LDPC del IEEE 802.11n (WiFi)

Un censo verificado a máquina de los grafos de control de paridad desplegados,
vía la ley del gap de fórmula de traza

Carles Marín*

Investigador independiente

karlesmarin@gmail.com

Junio de 2026

Resumen

Los ciclos cortos en el grafo de Tanner de un código de control de paridad de baja densidad (LDPC) degradan la decodificación por propagación de creencias, y contarlos es una tarea estándar en el diseño de códigos. Este artículo no los cuenta más rápido. Los cuenta *con un certificado*: para cada uno de los cuatro códigos LDPC del estándar IEEE 802.11n (WiFi) con longitud de bloque $n = 648$, el número de ciclos más cortos se obtiene de una identidad verificada a máquina —la ley del gap de fórmula de traza, demostrada libre de **sorry** en Lean 4 en el artículo gemelo [11]— y se contrasta con tres cálculos mutuamente independientes que coinciden exactamente: la traza non-backtracking $\text{tr } B^g/2g$, la ley del gap $(\text{tr } A^g - p_g)/2g$ y la enumeración directa. El censo da $c_6 = 3942$ (rate 1/2), $c_6 = 8046$ (rate 2/3), $c_4 = 54$ (rate 3/4, donde el cuello colapsa a 4) y $c_6 = 32\,346$ (rate 5/6), y cuenta una historia de diseño: el código de rate 3/4 paga su densidad con cuadrados, los ciclos más dañinos, mientras que el de rate 5/6 recupera su cuello al precio de treinta y dos mil hexágonos. Los algoritmos de conteo de la literatura de códigos son más rápidos y llegan más lejos; la contribución aquí es la capa de certificación —una identidad exacta respaldada por un teorema con prueba comprobable— que, según mi mejor conocimiento, ninguna herramienta de conteo previa lleva.

Mapa del lector. El teorema es prestado (Sección 2; demostrado en el gemelo). Lo nuevo es el censo auditado: tres rutas independientes (Sección 3), los cuatro códigos desplegados (Sección 4, Tabla 1) y la contabilidad honesta de lo que el certificado compra y no compra (Secciones 5–6).

1. Introducción: un número del que vive un decodificador

¿Qué significa *fiarse* de un conteo de ciclos? Cuando Gallager inventó los códigos LDPC en 1962 el mundo no les dio uso; treinta años después volvieron —al WiFi, al 5G, al espacio profundo— decodificados por propagación de creencias sobre un *grafo de Tanner* bipartito [2] cuyos dos lados son los bits del código y sus controles de paridad [1, 3]. La propagación de creencias es exacta sobre un árbol y solo aproximada sobre un grafo con ciclos: un ciclo corto deja que un mensaje vuelva a su origen y se refuerce a sí mismo, un rumor confundido con una confirmación independiente. Cuanto más corto el ciclo, más cerrado el lazo, así que el *cuello* (girth) del grafo de Tanner —la longitud de

*Censo computado con asistencia de IA (Claude, Anthropic); la matemática, el teorema sobre el que descansa y todas las afirmaciones son responsabilidad del autor. La identidad certificada se demuestra en el artículo gemelo [11].

su ciclo más corto— y el *número* de ciclos en el cuello y justo por encima se volvieron cantidades que un diseñador de códigos mide [2, 4].

Medirlas es un problema resuelto. Algoritmos espectrales y de paso de mensajes cuentan ciclos cortos con eficiencia, y sobre los grafos cuasi-cíclicos de los estándares reales cuentan bastante más allá del cuello [4, 5, 6]. Este artículo no añade nada a su velocidad ni a su alcance. Añade una cosa que les falta: un conteo cuya identidad definitoria es un *teorema con una prueba verificada a máquina*. El artículo gemelo [11] demuestra, libre de **sorry** en Lean 4 sobre Mathlib, que para un grafo finito de cuello g el número de ciclos más cortos es

$$c_g = \frac{\text{tr } A^g - p_g}{2g} = \frac{\text{tr } B^g}{2g},$$

con A la matriz de adyacencia, B el operador non-backtracking de Hashimoto y p_g la g -ésima suma de potencias de las raíces del polinomio de matchings. Aquí aplicamos esa identidad, como *censo certificado*, a los códigos LDPC que un chip de WiFi ejecuta de verdad.

2. La identidad que aplicamos, y hasta dónde está certificada

¿En qué teorema nos apoyamos, y dónde se detiene su garantía? El artículo gemelo [11] demuestra la *ley del gap de fórmula de traza*: para todo k en la ventana exacta $1 \leq k \leq g + 1$,

$$\text{tr } A^k - p_k = \text{tr } B^k.$$

Por debajo del cuello ($k < g$) ambos lados se anulan; en $k \in \{g, g + 1\}$ ambos valen $2k c_k$, donde c_k es el número de k -ciclos. Dos consecuencias son todo lo que necesitamos. Primera: la anulación bajo el cuello, $\text{tr } B^k = 0$ para $k < g$, es un certificado de cordura: un grafo de Tanner de cuello g tiene que reportar cero caminatas cerradas non-backtracking más cortas que g , y cualquier grafo de código que lo incumpla está mal interpretado. Segunda: en $k = g$ la ley es el conteo de ciclos más cortos de arriba.

¿Hasta dónde está certificada? Ambos hechos son teoremas en Lean, que dependen solo de los tres axiomas estándar (**propext**, **Classical.choice**, **Quot.sound**); la anulación bajo el cuello es la “Piedra A” y la igualdad en la ventana es el capstone “Piedra B” del gemelo. Así que la *identidad* que convierte espectros y conteos de matchings en un conteo de ciclos está verificada a máquina de extremo a extremo. Lo que *no* forma parte del teorema es la aritmética sobre una matriz concreta: evaluar $\text{tr } A^g$, p_g o $\text{tr } B^g$ sobre un grafo de Tanner concreto de 2376 aristas es cómputo exacto ordinario, no una prueba en Lean. Esa brecha —entre una fórmula certificada y su evaluación sobre una entrada concreta— es exactamente lo que el cruce de tres rutas de la sección siguiente está construido para cerrar.

3. Tres rutas a un mismo número

¿Por qué computar el mismo entero de tres maneras? Porque una *fórmula* certificada evaluada por código falible solo es tan fiable como la evaluación, y la guarda más barata contra un desliz de implementación es la redundancia por método independiente. El censo computa cada c_g por tres rutas que no comparten ninguna ruta de código y se encuentran solo en la respuesta.

Las tres rutas (mutuamente independientes)

R1 — traza non-backtracking. Constrúyase el operador de Hashimoto B sobre los $2|E|$ dardos como matriz dispersa 0/1 y fórmese $\text{tr } B^g$ sin densificar nunca (potencias por productos dispersos, trazas por la identidad de Hadamard $\text{tr } B^k = \sum (B^a \circ (B^b)^\top)$, $a + b = k$); entonces $c_g = \text{tr } B^g / 2g$.

R2 — la ley del gap. Calcúlese $\text{tr } A^g$ desde la matriz de adyacencia y p_g por Newton desde conteos de matchings — $p_4 = 2m_1^2 - 4m_2$, $p_6 = 2m_1^3 - 6m_1m_2 + 6m_3$, con $m_1 = |E|$, $m_2 = \binom{m_1}{2} - \sum_v \binom{d_v}{2}$, y m_3 por una recursión exacta de borrado de aristas bipartita—; entonces $c_g = (\text{tr } A^g - p_g) / 2g$.

R3 — enumeración directa. Cuéntense los ciclos combinatoriamente: para $g = 4$, súpese $\binom{\text{codeg}}{2}$ sobre pares de vértices del mismo lado (cuatro-ciclos bipartitos); para $g = 6$, una búsqueda de subgrafo VF2 de C_6 , dividida por sus 12 enraizamientos automorfos.

R1 vive en el mundo geodésico del operador non-backtracking; R2 en el mundo espectral-más-matchings de la ley del gap; R3 en la enumeración llana. Invocan bibliotecas disjuntas y aritmética disjunta. Cuando las tres devuelven el mismo entero, un error tendría que ser una *coincidencia entre tres métodos* —y la ley del gap es justo el teorema que dice que R1 y R2 deben coincidir, así que un acuerdo a tres bandas también reconfirma la identidad en cada grafo real.

La ruta de conteo de matchings merece una observación. La recursión $m_3 = \frac{1}{3} \sum_{(u,v) \in E} m_2(G - \{u,v\})$ se evalúa en forma cerrada sobre el grafo bipartito — $N(u)$ y $N(v)$ son disjuntos, así que los números de matchings del borrado se reducen a sumas de grados— sin copiar nunca el grafo; esto es lo que mantiene R2 en unos segundos en vez de unas horas sobre una instancia de 2376 aristas.

4. El censo: cuatro códigos que un chip de WiFi ejecuta

¿Qué aspecto tienen de verdad los códigos desplegados? El IEEE 802.11n especifica códigos LDPC en tres longitudes de bloque; tomamos la mayor, $n = 648$, con tamaño de lifting cuasi-cíclico $z = 27$, en las cuatro rates. Cada matriz de control de paridad da un grafo de Tanner sobre $|V| = n + m$ vértices ($m = n(1 - \text{rate})$ controles) con un número constante $|E| = 2376$ de aristas. El censo, con los ceros bajo el cuello confirmados en cada código, es la Tabla 1.

rate	$ V $	$ E $	cuello g	c_g (R1 = R2 = R3)	ciclo más corto
1/2	972	2376	6	3 942	hexágono
2/3	864	2376	6	8 046	hexágono
3/4	810	2376	4	54	cuadrado
5/6	756	2376	6	32 346	hexágono

Cuadro 1: El censo certificado de ciclos cortos de los códigos LDPC del IEEE 802.11n, $n = 648$, $z = 27$. Para cada código las tres rutas independientes de la Sección 3 devuelven el mismo entero, y la anulación bajo el cuello certificada en Lean, $\text{tr } B^k = 0$ ($k < g$), vale en los cuatro grafos de Tanner. Computado en aritmética exacta; la *pipeline* es `research/ldpc-gaplaw/census_pilot2.py`.

La historia que cuentan los cuatro números. Léase la tabla hacia abajo y aparece un compromiso de diseño (Figura 1). En rate 1/2 el grafo es bastante disperso para sostener cuello 6 con los menos hexágonos, 3942 —el más limpio de los cuatro—. En rate 2/3 el cuello sobrevive pero la población de hexágonos se duplica. En rate 3/4 el cuello *colapsa* a 4: la densidad fuerza 54 cuadrados, los ciclos más dañinos que un grafo de Tanner puede llevar. En rate 5/6, aún más denso, los diseñadores

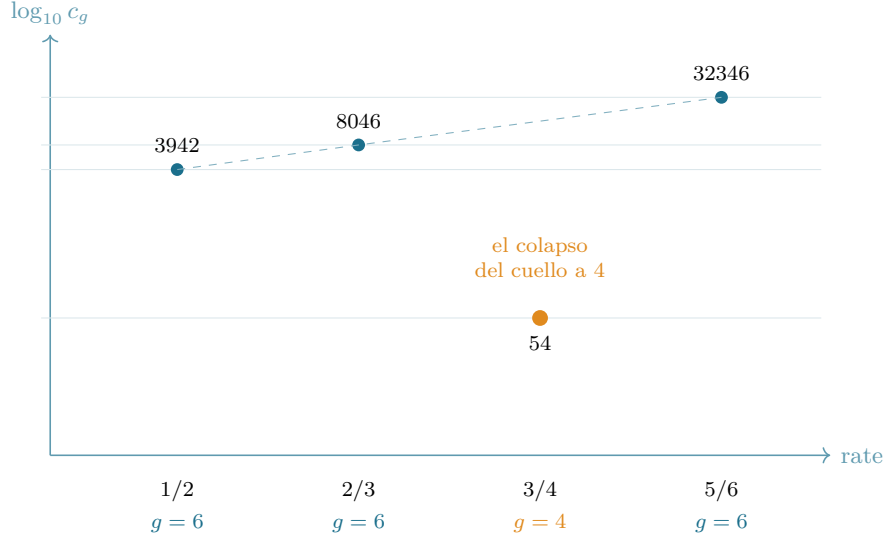


Figura 1: El paisaje de diseño de los cuatro códigos del IEEE 802.11n en $n = 648$. Las tres rates de cuello 6 trazan un conteo creciente de hexágonos (discontinua); rate 3/4 se sale de esa tendencia porque su cuello colapsa a 4, donde el conteo es pequeño (54) pero los ciclos son del tipo corto y dañino para el decodificador. “Pocos ciclos” y “ciclos largos” son bienes distintos, y el censo los separa.

recuperan cuello 6 —intercambiando los cuadrados— pero lo pagan con 32 346 hexágonos, un salto de ocho veces sobre rate 1/2. El certificado no juzga el intercambio; hace exacto su precio.

5. Qué compra el certificado

Contar ciclos cortos está resuelto —así que ¿qué hay de nuevo aquí? No el conteo, ni su coste. El método espectral de Blake y Lin y los de paso de mensajes de Karimi y Banihashemi cuentan más lejos (hasta $2g - 2$ y más allá, con términos de corrección) y más rápido, y explotan la estructura de bloques cuasi-cíclica que esta *pipeline* ignora [5, 4, 6]. Frente a ellos este censo es más lento y de menor alcance.

Lo que lleva y ellos no es un *certificado*: la identidad que produce el número es un teorema con una prueba que una máquina ha comprobado, no una derivación confiada a un papel. De ahí se siguen tres cosas que importan en la práctica. La anulación bajo el cuello es una *comprobación de integridad gratuita* sobre una matriz de control de paridad ya leída: una sola entrada mal leída que cree un ciclo corto fantasma aparece como un $\text{tr } B^k$ no nulo donde el teorema prohíbe uno. El acuerdo de las rutas non-backtracking y de matchings no es una coincidencia afortunada sino una igualdad *predicha*, así que un desajuste localiza el fallo en una ruta. Y los momentos más pequeños del operador non-backtracking, $\text{tr } B^g = 2g c_g$ y $\text{tr } B^{g+1} = 2(g+1)c_{g+1}$, que gobiernan los umbrales de percolación y epidémicos en una red [7] y el límite de detectabilidad en detección de comunidades [8], quedan ahora fijados exactamente desde cantidades computadas por código independiente y bien probado —una prueba de regresión de nivel de entrada para cualquier implementación de B .

6. Fronteras honestas

¿Dónde se detiene exactamente la garantía? En tres sitios, dichos sin rodeos.

La ventana es $k \leq g + 1$. La identidad certificada cuenta ciclos *en* el cuello (y en $g + 1$); las cantidades de ingeniería más allá —multiplicidades hasta $2g - 2$, los términos de corrección de [5, 6]— no están cubiertas por el teorema y no se computan aquí.

Las matrices son de terceros. Los grafos de Tanner se construyen desde matrices de control de paridad derivadas del estándar IEEE 802.11n vía una herramienta pública, no re-derivadas del Anexo F del estándar. Su estructura de bloques cuasi-cíclica ($z = 27$) y sus perfiles de grados se cruzaron, y el acuerdo a tres rutas es en sí mismo fuerte evidencia de que los grafos son los pretendidos, pero un censo de grado de estándar re-derivaría H del documento normativo. El certificado cubre la *identidad de conteo*, no la procedencia de la entrada.

Sin novedad algorítmica. Las rutas son de manual. La contribución es su *composición bajo una identidad verificada a máquina*, que una búsqueda en la literatura del 2026-06-11 halló ausente: ningún trabajo en Lean, Coq u otro asistente de pruebas sobre conteo de ciclos para teoría de códigos. Como siempre, tal afirmación se hace solo según el mejor conocimiento de uno.

7. Preguntas abiertas

1. *El titular 5G.* La misma *pipeline* se aplica a los códigos NR de 3GPP (TS 38.212, grafos base BG1 y BG2 con sus tamaños de lifting desplegados). Un censo certificado a lo largo de la familia rate-compatible de 5G es el siguiente objetivo natural —grafos mayores, el mismo teorema.
2. *La ventana corregida en Lean.* Los ingenieros cuentan hasta $2g - 2$ con correcciones en forma cerrada; el artículo gemelo deja el primer término de corrección ($k = g + 2$, las caminatas piruleta) como formalización abierta. Un censo certificado que alcance $2g - 2$ igualaría el alcance de la literatura *con* una prueba.
3. *Re-derivación desde el estándar.* Construir H directamente del Anexo F del IEEE 802.11n y de la TS 38.212 de 3GPP, dentro de la *pipeline* certificada, cerraría la brecha de procedencia y haría el censo de grado de estándar de extremo a extremo.
4. *El certificado como instrumento de diseño.* El par $(\text{tr } B^g, \text{tr } B^{g+1})$ es un invariante certificado y barato de un grafo de Tanner. ¿Cuánto del perfil de ciclos cortos de un código —y de su comportamiento de decodificación— fija este par certificado, y dónde deja de distinguir códigos que decodifican distinto?

Disponibilidad de datos y código

Los archivos alist de control de paridad, la *pipeline* del censo (`research/ldpc-gaplaw/census_pilot2.py`) y la salida completa están en el repositorio público <https://github.com/karlesmarin/godsil-gutman-lean>, junto a las fuentes Lean del artículo gemelo [11] que demuestran la identidad aquí aplicada.

Referencias

- [1] R. G. Gallager, *Low-density parity-check codes*, IRE Trans. Inform. Theory **8** (1962) 21–28.

- [2] R. M. Tanner, *A recursive approach to low complexity codes*, IEEE Trans. Inform. Theory **27** (1981) 533–547.
- [3] D. J. C. MacKay, *Good error-correcting codes based on very sparse matrices*, IEEE Trans. Inform. Theory **45** (1999) 399–431.
- [4] M. Karimi, A. H. Banihashemi, *Message-passing algorithms for counting short cycles in a graph*, IEEE Trans. Commun. **61** (2013) 485–495.
- [5] I. F. Blake, S. Lin, *On short cycle enumeration in biregular bipartite graphs*, IEEE Trans. Inform. Theory **64** (2018) 6526–6535.
- [6] A. Dehghan, A. H. Banihashemi, *On computing the multiplicity of cycles in bipartite graphs using the degree distribution and the spectrum of the graph*, IEEE Trans. Inform. Theory **65** (2019) 3778–3789.
- [7] B. Karrer, M. E. J. Newman, L. Zdeborová, *Percolation on sparse networks*, Phys. Rev. Lett. **113** (2014) 208702.
- [8] F. Krzakala, C. Moore, E. Mossel, J. Neeman, A. Sly, L. Zdeborová, P. Zhang, *Spectral redemption in clustering sparse networks*, Proc. Natl. Acad. Sci. **110** (2013) 20935–20940.
- [9] K. Hashimoto, *Zeta functions of finite graphs and representations of p -adic groups*, Adv. Stud. Pure Math. **15** (1989) 211–280.
- [10] The mathlib community, *The Lean mathematical library*, en: CPP 2020, ACM, 2020, 367–381.
- [11] C. Marín, *The walks that remember the cycles: a machine-checked sharp gap law between the matching polynomial and the non-backtracking spectrum in Lean 4* (Parte VI de la serie godsil–gutman–lean), Zenodo, 2026.